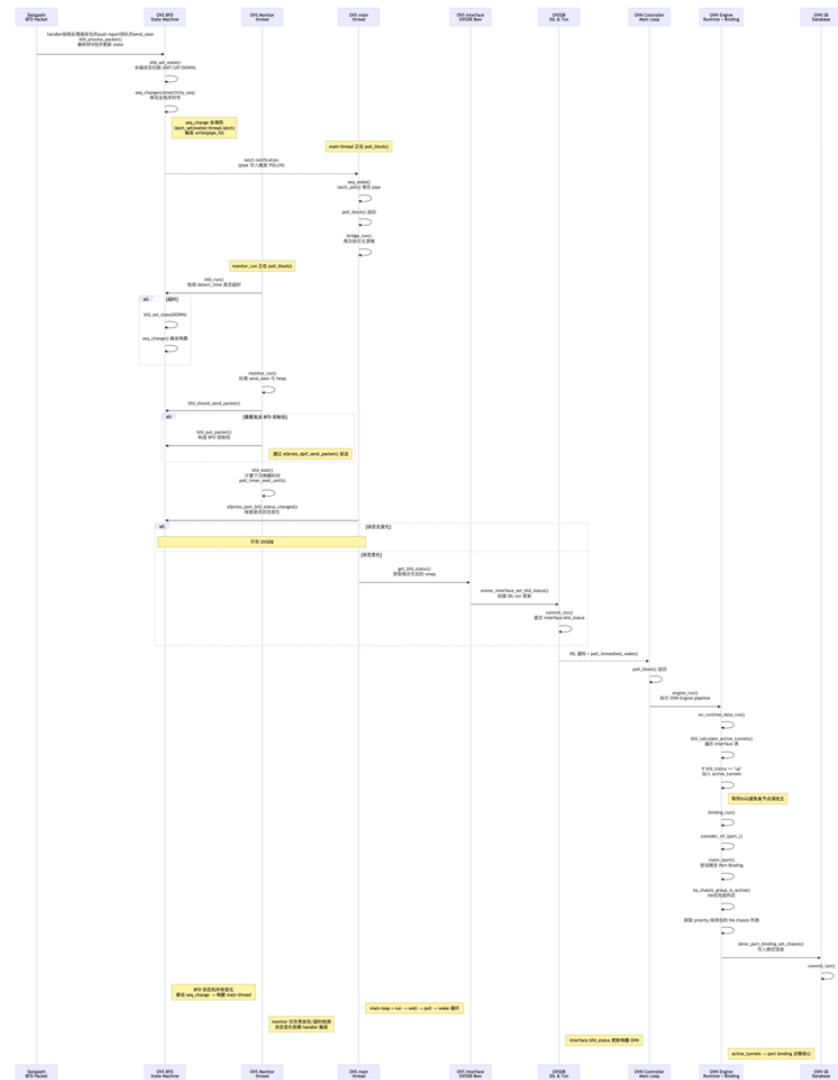


BFD状态变化影响流程



bfd状态变化影响流程

BFD状态改变唤醒主线程的代码过程：

```

1 handler线程：
2 process_special
3     -> bfd_should_process_flow(xport->bfd, flow, wc)
4     -> bfd_process_packet(xport->bfd, flow, packet)
5     -> bfd_set_state
6         -> bfd_status_changed(bfd)
7         -> seq_change(connectivity_seq_get())
8         -> seq_change_protected(seq)
9         -> atomic_store_explicit(&seq->value, seq_value, memory_order_relaxed)
10        -> seq_wake_waiters
11        -> HMAP_FOR_EACH_SAFE (waiter, hmap_node, &seq->waiters)
12            latch_set(&waiter->thread->latch);
13            -> ignore(write(latch->fds[1], "", 1))

```

```

14         seq_waiter_destroy(waiter);
15     }
16     -> bfd->status_changed = true
17     -> ofproto_dpif_monitor_port_send_soon(xport->ofport)
18     -> guarded_list_push_back(&send_soon, &entry->list_node, SIZE_MAX)
19
20 main线程:
21     main
22     -> while (!exit_args.exiting) {
23         -> bridge_run()
24         -> run_status_update
25             -> if (seq != connectivity_seqno || status_txn_try_again
26                 -> HMAP_FOR_EACH (br, node, &all_bridges)
27                 -> HMAP_FOR_EACH (port, hmap_node, &br->ports)
28                 -> LIST_FOR_EACH (iface, port_elem, &port->ifaces)
29                 -> iface_refresh_ofproto_status(iface)
30                 -> if ofproto_port_bfd_status_changed
31                 -> ofproto_port_get_bfd_status
32                 -> ovsrec_interface_set_bfd_status
33             -> ovsdb_idl_txn_commit(status_txn)
34         -> bridge_wait()
35         -> status_update_wait()
36         -> seq_wait(connectivity_seq_get(), connectivity_seqno)
37             -> if (waiter->value != value) // 如果handler线程修改了b
38             -> poll_immediate_wake_at
39             -> if (!waiter->thread->waiting) // 如果handler线程未改变
40             -> latch_wait_at(&waiter->thread->latch, where)
41             -> poll_fd_wait_at(latch->fds[0], POLLIN, where)
42             -> poll_create_node(fd, 0, events, where)
43         -> poll_block() // 主线程阻塞等待
44         -> seq_woke() // seq改变唤醒
45         -> seq_thread_woke(thread)
46         -> latch_poll(&thread->latch) // pipe可读,即seq_change已
47         -> do {
48             ret = read(latch->fds[0], &latch_buffer, size
49             result |= ret > 0
50             } while (ret == sizeof latch_buffer);
51     }
52
53 monitor线程:
54     monitor_main
55     -> while (!latch_is_set(&monitor_exit_latch)) {
56         -> monitor_run()
57             -> monitor_check_send_soon(&packet) // 处理handler push到send_s
58             -> while (!guarded_list_is_empty(&send_soon)) {
59                 entry = CONTAINER_OF(guarded_list_pop_front(&send_soon)
60                 mport = mport_find(entry->ofport)
61                 monitor_mport_run(mport, packet)
62             }
63             -> while (!heap_is_empty(&monitor_heap) { // 处理heap中到了时间的
64                 mport = CONTAINER_OF(heap_max(&monitor_heap), struct mpor
65                 monitor_mport_run(mport, &packet)
66                 -> if bfd_should_send_packet(mport->bfd)
67                 -> bfd_put_packet(mport->bfd, packet, mport->hw_addr,
68                 -> bfd->last_tx = time_msec()
69                 -> bfd_set_next_tx(bfd)
70                 -> ofproto_dpif_send_packet(mport->ofport, oam, packe
71             }
72             -> monitor_check_send_soon(&packet) // 处理handler push到send_s
73             -> poll_timer_wait_until(MIN(next_timeout, next_mport_wakeup))
74             -> latch_wait(&monitor_exit_latch)
75             -> poll_block()
76     }

```

Mermaid流程图源码：

```

1 sequenceDiagram
2     participant DP as Datapath<br/>BFD Packet
3     participant BFD as OVS BFD<br/>State Machine
4     participant MON as OVS Monitor<br/>thread
5     participant MAIN as OVS main<br/>thread

```

```

6 participant IFACE as OVS Interface<br/>OVSDb Row
7 participant OVSDb as OVSDb<br/>IDL & Txn
8 participant CTRL as OVN Controller<br/>Main Loop
9 participant ENG as OVN Engine<br/>Runtime + Binding
10 participant SBDB as OVN SB<br/>Database
11
12 %% =====
13 %% 阶段1: BFD包接收 (数据面 -> handler thread)
14 %% =====
15 DP->>BFD: handler线程处理接收包并push mport到队列send_soon<br/>bfd_pro
16 BFD->>BFD: bfd_set_state()<br/>本端状态切换 (INIT/UP/DOWN)
17
18 BFD->>BFD: seq_change(connectivity_seq)<br/>修改全局序列号
19 note right of BFD: seq_change 会调用<br/>latch_set(waiter.thread.la
20
21 %% =====
22 %% 阶段2: main-thread 被 seq/latch 唤醒
23 %% =====
24 note over MAIN: main-thread 正在 poll_block()
25
26 BFD->>MAIN: latch notification<br/>(pipe 写入触发 POLLIN)
27 MAIN->>MAIN: seq_woke()<br/>latch_poll() 清空 pipe
28 MAIN->>MAIN: poll_block() 返回
29
30 MAIN->>MAIN: bridge_run()<br/>再次执行主逻辑
31
32 %% =====
33 %% 阶段3: Monitor 定时任务 (每100ms)
34 %% =====
35 note over MON: monitor_run 正在 poll_block()
36
37 MON->>BFD: bfd_run()<br/>检测 detect_time 是否超时
38 alt 超时
39     BFD->>BFD: bfd_set_state(DOWN)
40     BFD->>BFD: seq_change() 触发唤醒
41 end
42
43 MON->>MON: monitor_run()<br/>处理 send_soon 与 heap
44 MON->>BFD: bfd_should_send_packet()
45 alt 需要发送 BFD 控制包
46     MON->>BFD: bfd_put_packet()<br/>构造 BFD 控制包
47     note right of MON: 通过 ofproto_dpif_send_packet() 发送
48 end
49
50 MON->>MON: bfd_wait()<br/>计算下次唤醒时间<br/>poll_timer_wait_until(
51
52 %% =====
53 %% 阶段4: 状态变化 -> 写入 OVSDb Interface.bfd_status
54 %% =====
55 MAIN->>BFD: ofproto_port_bfd_status_changed()<br/>检查是否状态变化
56 alt 状态无变化
57     note over MAIN,BFD: 不写 OVSDb
58 else 状态变化
59     MAIN->>IFACE: get_bfd_status()<br/>获取格式化后的 smap
60     IFACE->>OVSDb: ovsrec_interface_set_bfd_status()<br/>创建 IDL t
61     OVSDb->>OVSDb: commit_txn()<br/>提交 Interface.bfd_status
62 end
63
64 %% =====
65 %% 阶段5: OVSDb 变更唤醒 OVN 主循环(ctrl)
66 %% =====
67 OVSDb->>CTRL: IDL 通知 + poll_immediate_wake()
68 CTRL->>CTRL: poll_block() 返回
69
70 CTRL->>ENG: engine_run()<br/>执行 OVN Engine pipeline
71
72 %% =====
73 %% 阶段6: OVN Engine: BFD 影响 active_tunnels
74 %% =====
75 ENG->>ENG: en_runtime_data_run()
76 ENG->>ENG: bfd_calculate_active_tunnels()<br/>遍历 Interface 表

```

```
77
78     ENG->>ENG: if bfd_status == "up"<br/>加入 active_tunnels
79
80     note right of ENG: 等待2s以避免备节点误抢主
81
82     %% =====
83     %% 阶段7: 端口绑定 Claim 流程
84     %% =====
85     ENG->>ENG: binding_run()
86     ENG->>ENG: consider_vif_lport_()
87     ENG->>ENG: claim_lport()<br/>尝试绑定 Port Binding
88
89     ENG->>ENG: ha_chassis_group_is_active()<br/>HA优先级判定
90     ENG->>ENG: 获取 priority 排序后的 HA chassis 列表
91
92     ENG->>SBDB: sbrec_port_binding_set_chassis()<br/>写入绑定信息
93     SBDB->>SBDB: commit_txn()
94
95     %% =====
96     %% 小结关键点
97     %% =====
98     note right of BFD: BFD 状态机所有变化<br/>都会 seq_change → 唤醒 main-
99     note right of MAIN: main-loop = run → wait → poll → wake 循环
100    note right of MON: monitor 仅负责发包/超时检测<br/>状态变化依赖 handler
101    note right of OVSDB: Interface.bfd_status 更新唤醒 OVN
102    note right of ENG: active_tunnels → port binding 决策核心
```